

Introduction to testing

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

What is testing?

Software Testing:

- Process of evaluating software to determine whether it meets specified requirements

Testing can be:

- Manual - necessary but limited and less reliable
- Automated - powerful way to execute hundreds of tests and checks in seconds

We'll use JUnit 5 for automated testing



Edge cases

"If it's not tested, it's broken."¹

Edge case - input or condition that is at the extreme or boundary of what is considered typical or expected

Examples: variables at maximum capacity, null or empty variables, negative values



¹ Quote originally by Bruce Eckel

Example of unexpected behavior at an edge case

Consider the following code:

```
public int addTwoNumbers(int a, int b) {  
    return a + b;  
}
```

⚠ Consider inputs `2147483647` (integer max value) and `1`

➡ The output is `-2147483648` (integer min value)

Recall: Variable sizes - primitives have limited size

	Primitive type	Value	Example
whole numbers	byte	-128 to 128	<code>byte age = 5;</code>
	short	-32,768 to 32,767	<code>short sqFeet = 3240;</code>
	int	-2,147,483 and 2,147,647	<code>int accountBalanace = 100000;</code>
	long	-2^63 and 2^63	<code>long nationalDebt = 34849081491943L;</code>
decimals	float	3.4e-038 to 3.4e+038	<code>float unitPrice = 4.99f</code>
	double	1.7e-308 to 1.7e+308	<code>double pi = 3.141592653589793;</code>
	char	alphabet, punc, numbers, etc.	<code>char grade = 'C';</code>
	boolean	true or false	<code>boolean isZoned = true;</code>

Example: Ariane 5 launch

First flight of the Ariane 5 rocket in 1996:

- Spent over 8 years in development
- Cost over \$370 million
- Exploded 37 seconds after launch
- Explosion was caused by software error:
short overflow



Benefits of testing

- Guards against edge case bugs
- Leads to better understanding of the project and its requirements
- Speeds up development in the long run
- Automates enforcing requirements
- Guards against human error



¹ https://books.google.com/books/about/Clean_Architecture.html?id=8ngAkAEACAAJ

Let's practice!

INTRODUCTION TO TESTING IN JAVA

Introduction to JUnit

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

Quick facts

- JUnit is the most popular Java testing library
- It is well-maintained and constantly updated



JUnit 5

Writing a JUnit test

Let's write a test for the `addTwoNumbers()` method:

```
public int addTwoNumbers(int a, int b) {  
    return a + b;  
}
```

JUnit @Test annotation

- Install JUnit using an IDE or build tools (example in the citation)
- Every test must have the `@Test` annotation:

```
import org.junit.jupiter.api.Test;  
  
@Test  
// Test method will be here
```

Annotation - special metadata that indicates how the method should be handled by compilers and frameworks. Annotations begin with `@`

¹ <https://www.jetbrains.com/help/idea/junit.html>

JUnit test structure

```
import org.junit.jupiter.api.Test;

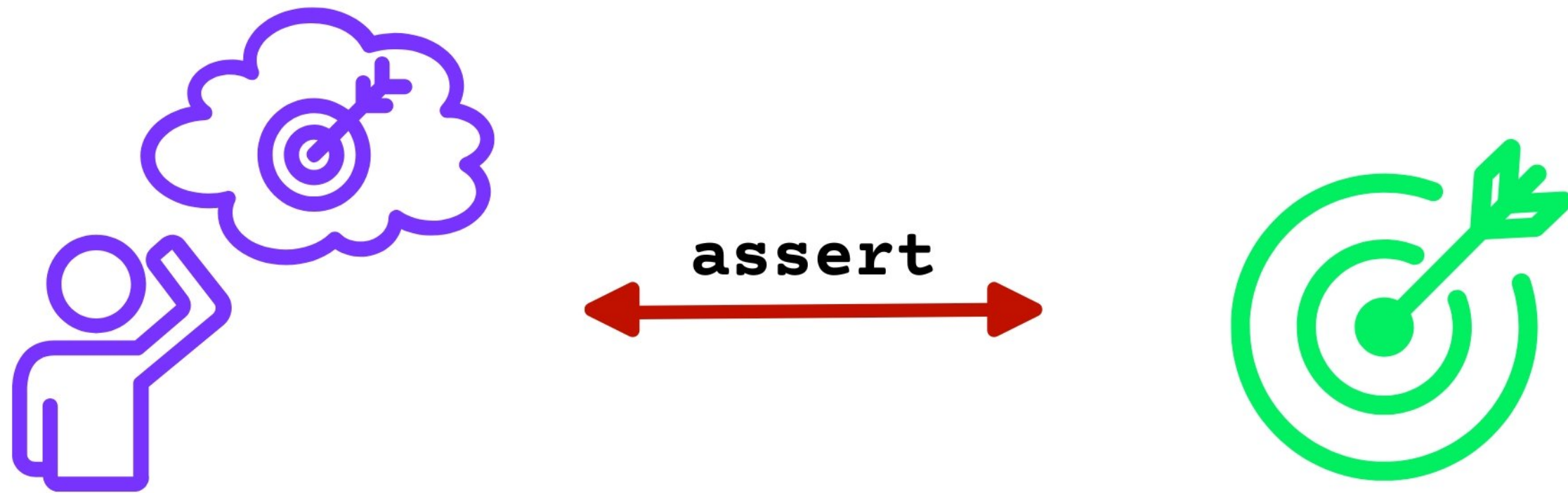
@Test
void testAddTwoNumbers() {
    // Arrange - Given
    int num1 = 2;
    int num2 = 2;

    // Act - When
    int actual = addTwoNumbers(num1, num2);

    // Assert - Then
    assertEquals(4, actual);
}
```

Assert statements

- Quick way to compare **actual** outcomes to **expected** outcomes
- If expected and actual outcomes don't match, test fails
- Help quickly catch bugs and errors



Test outcomes

Consider the assertion:

```
assertEquals(4, actual); // Will succeed if actual == 4
```

If successful, will produce a `Test passed` message

If the values are not equal, e.g. `actual = 5`:

```
org.opentest4j.AssertionFailedError: expected: <4> but was: <5>
```

*Remember argument order! First argument is the **expected** one in JUnit.*

Additional test for overflow

```
@Test
void testAddTwoNumbers() {
    // Given
    int num1 = 2147483647;
    int num2 = 1;

    // When
    int actual = addTwoNumbers(num1, num2);

    // Then
    assertEquals(-2147483648, actual);
}
```


Notes about JUnit and testing

- There can be multiple assertions per test
- All assertions need to succeed in order for the test to pass
- We can import all assertions using:

```
import org.junit.jupiter.api.Assertions.*;
```

More notes

- Following is a custom Datacamp tool, not needed in IDE:

```
import static com.datacamp.util.testing.CustomJUnitTestLauncher.launchTestsAndPrint;
```

- `import static package.Class.method` allows using `method` as just `method`, without writing `package.Class.method`

```
import static java.lang.Math.max;
```

```
...
```

```
max(3, 5); // Instead of Math.max(3, 5)
```

Let's practice!

INTRODUCTION TO TESTING IN JAVA

Unit testing with JUnit

INTRODUCTION TO TESTING IN JAVA



Maria Milusheva
Senior Software Engineer

Unit testing - definition

Two key concepts:

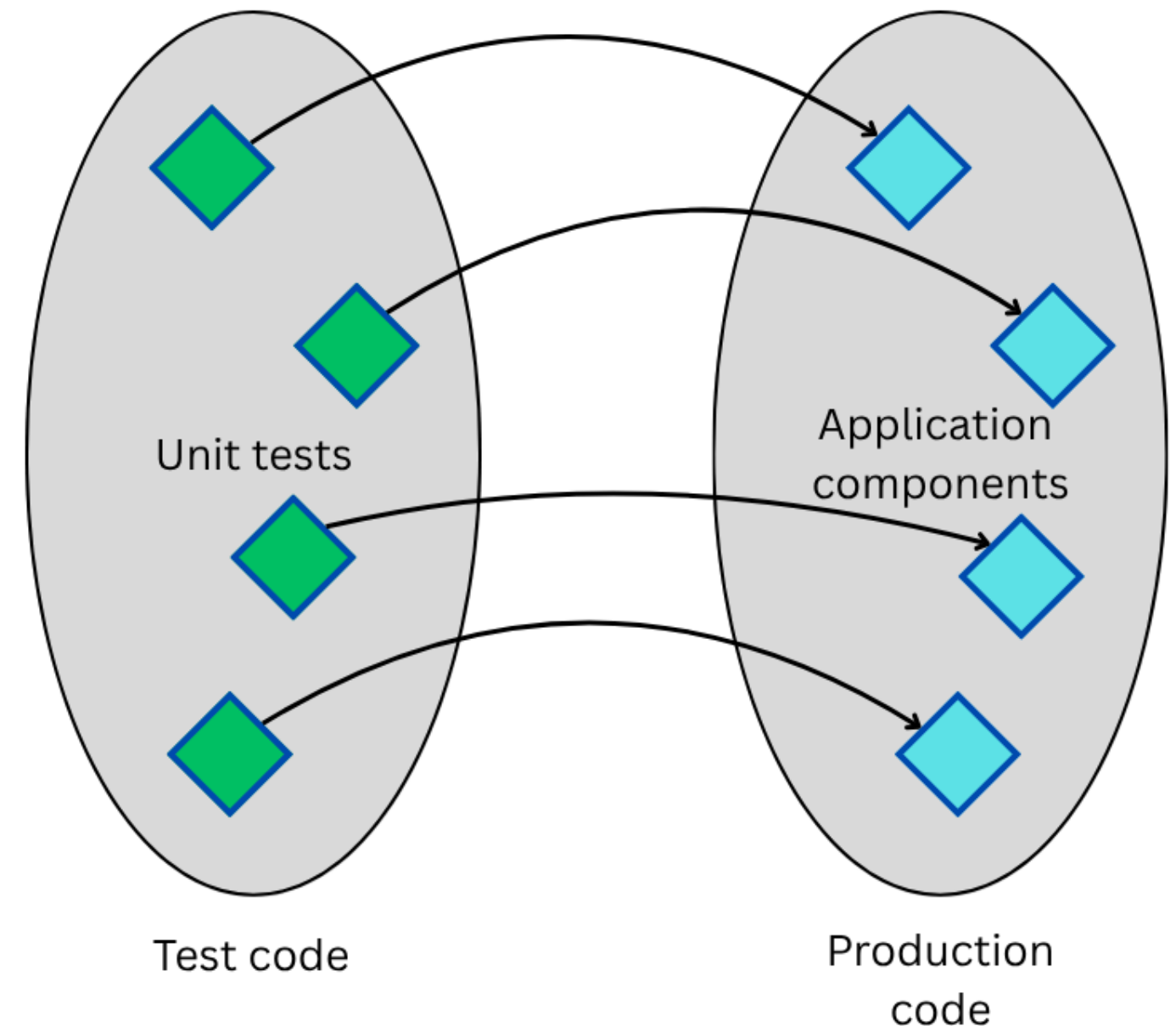
Unit - the smallest testable part of an application, such as a method

Unit testing - A test that focuses on verifying the correctness of a single "unit" of code in isolation from other parts of the application

Unit testing is about details

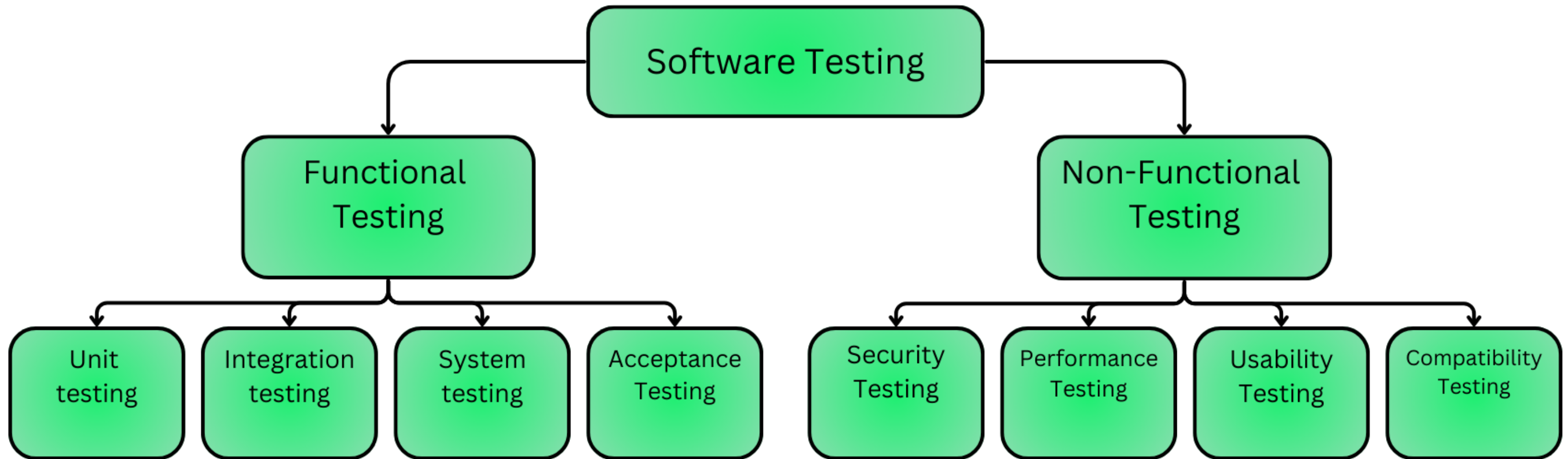
Notes:

- Unit testing is about small pieces of logic, not the big picture
- Other types of testing focus on the big picture (e.g. integration)
- JUnit gets its name because its primary use case is unit testing



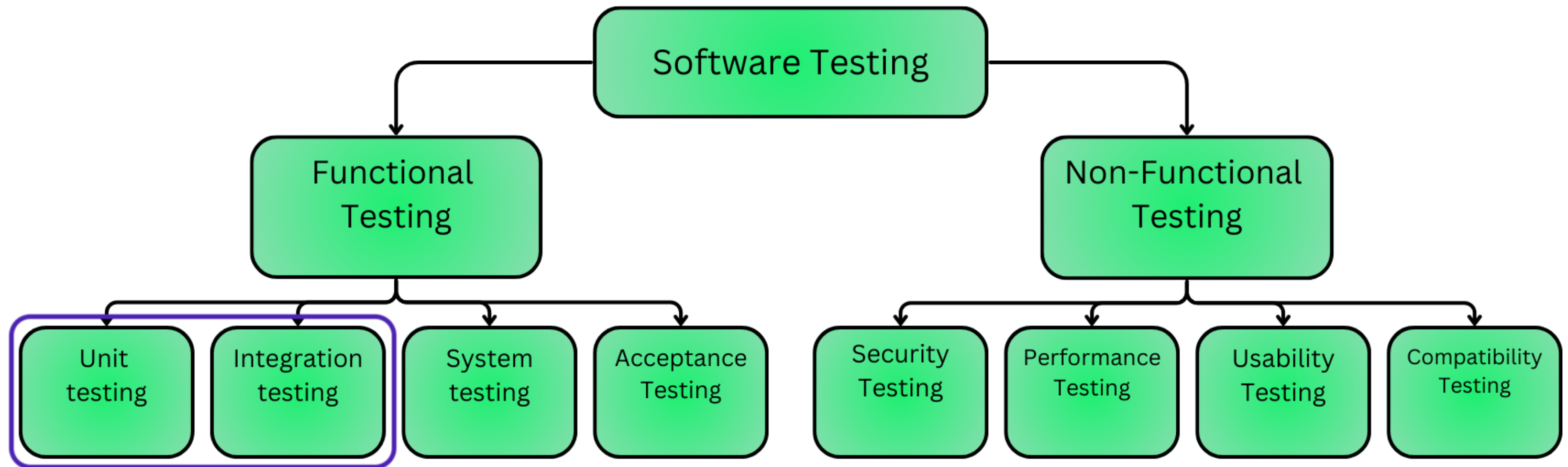
Types of testing - brief overview

- In this course we will discuss mostly **unit** and **integration** tests
- But there are many types of testing!



Types of testing - brief overview

- In this course we will discuss mostly **unit** and **integration** tests
- But there are many types of testing!



JUnit syntax: assertTrue() and assertFalse()

- `assertTrue()` checks that a statement is true:

```
List<String> list = new ArrayList<>();  
  
assertTrue(list.isEmpty()); // No error
```

- `assertFalse()` checks the opposite:

```
List<String> list = new ArrayList<>();  
list.add("A");  
  
assertFalse(list.isEmpty()); // No error
```

assertNull() and assertNotNull()

`null` variables can lead to `NullPointerException` !

- Assertions that check a variable is or isn't `null` - `assertNull()` and `assertNotNull()`
- Common use case: checking a value retrieved from somewhere is not null

```
Map<String, Integer> catalogue = new HashMap<>();
```

```
catalogue.put("item1", 10);
```

```
// No errors
```

```
assertNotNull(catalogue.get("item1"));
```

```
assertNull(catalogue.get("item2"));
```

Asserting exceptions

Suppose we want to verify an `ArrayIndexOutOfBoundsException` is thrown:

```
public String getIndex(String[] array, int index) {  
    return array[index];  
}
```

JUnit provides multiple ways to do this!

assertInstanceOf()

- JUnit provides `assertThrows()`, but it uses advanced Java syntax (lambda expressions)
- Can use `assertInstanceOf()` like this:

```
try {  
    getIndex(new String[] {}, 4);  
} catch (Exception e) {  
    // Pass the expected class of the exception and the exception itself  
    assertInstanceOf(ArrayIndexOutOfBoundsException.class, e);  
}
```

¹ <https://www.baeldung.com/junit-assert-exception>

Naming unit tests

- A typical project contains hundreds of unit tests
- They should be lightweight and simple to understand
- They should have informative names so that we can see at first glance what went wrong if one of them fails

For example: `methodUnderTest_expectedBehavior_conditions()`

¹ <https://dzone.com/articles/7-popular-unit-test-naming?fromrel=true>

Let's practice!

INTRODUCTION TO TESTING IN JAVA